

Semantic Analysis

A dark blue diagonal gradient bar that starts from the bottom left corner and extends towards the top right corner, covering the lower half of the slide.

Position of Semantic Analysis

- Semantic analysis is the third phase of a compiler
- It comes after syntax analysis and before intermediate code generation
- It checks the meaning of the program, not just structure

What is Semantic Analysis?

- Semantic analysis ensures the program is meaningful
- It verifies consistency of declarations and usage
- Detects errors that syntax analysis cannot detect

Syntax Error vs Semantic Error

- Syntax errors violate grammar rules
- Semantic errors violate meaning or logic of the program
- Semantic errors occur in syntactically correct programs

Main Tasks of Semantic Analysis

- Type checking
- Symbol table management
- Scope checking
- Function and procedure checking

Symbol Table

- A symbol table stores information about identifiers
- Stores name, type, scope, memory location
- Used during semantic analysis and code generation

Scope Handling

- Scope defines the visibility of variables
- Types of scope: global, local, block
- Semantic analyzer ensures correct scope usage

Type Checking

- Ensures operands are of compatible types
- Detects type mismatch errors
- Performed mostly at compile time

Static vs Dynamic Type Checking

- Static: type checked at compile time (C, Java)
- Dynamic: type checked at runtime (Python)
- Static type checking improves safety

Function Semantic Checking

- Checks number of parameters
- Checks parameter types
- Checks return type correctness

Control Statement Checking

- Conditions must evaluate to boolean
- Checks validity of break and continue
- Ensures return statement correctness

Error Detection in Semantic Analysis

- Undeclared variable usage
- Multiple declarations
- Type mismatch in expressions
- Wrong function calls

Summary

- Semantic analysis checks program meaning
- Uses symbol table and type rules
- Plays a crucial role before code generation